

医疗大模型新人快速上手指南

读者定位： 本指南面向刚进入医疗大模型/医学 AI 科研团队、医学背景为主、编程基础薄弱、缺少大模型使用经验的新同学。

使用目标： 帮助你快速理解 Python 环境配置、国内大模型 API 调用、AI 辅助编程、Debug 提问方法和 AI 科研绘图避坑。

1. 读者定位

本指南默认读者是：

刚进组
有一些医学背景
编程基础薄弱
没有系统使用过大模型 API
不了解 GitHub / Markdown / LaTeX

阅读路线图

本指南篇幅较长，不同需求的读者可以按以下路线选择性阅读：

你的情况	推荐路线
今天是第一天，电脑上什么都没装	从头到尾读 2.1 → 边读边操作
环境已配好，想学 API 调用	直接跳到 2.2 → 跟着创建第一个 API 脚本
代码跑不通，卡在报错了	跳到 第 3 章 → 用 Debug 模板向 AI 提问
在写论文、画流程图	跳到 第 4 章 → 学 AI 绘图 Prompt 写法

新手建议： 即使你想尽快完成项目，也至少完整阅读 2.1 节。环境配置是所有后续步骤的地基，这一步走稳了，后面会省很多时间。

2. 工具篇

2.1 如何快速搞定 Python 环境配置

对于医学背景新人来说，Python 环境配置的目标不是“学会所有编程知识”，而是先做到：

能打开 VS Code
能在终端输入命令
能安装 Python 包
能运行 .py 文件
能运行 Jupyter Notebook
能看懂最常见报错

2.1.1 先理解几个基础概念

名词	新人理解
Python	运行科研代码的语言
VS Code	写代码、管理项目文件的工具
Terminal / 终端	输入命令的地方
pip	安装 Python 第三方库的工具
Python package / 包	别人写好的功能模块，例如 pandas、sklearn
.py 文件	Python 脚本文件，适合运行固定流程
.ipynb 文件	Jupyter Notebook 文件，适合边分析边看结果
virtual environment	独立环境，避免不同项目依赖互相冲突

可以把这些工具理解成：

Python = 发动机
VS Code = 操作台
终端 = 下命令的窗口
pip = 安装工具
第三方库 = 试剂盒
项目文件夹 = 实验台

2.1.2 推荐安装路线

对新手来说，推荐最简单稳定的路线：

Python 官方版 + VS Code + Git

推荐安装顺序：

1. 安装 Python
2. 安装 VS Code
3. 安装 Git
4. 在 VS Code 中安装 Python 插件
5. 打开项目文件夹
6. 在终端中安装依赖
7. 运行测试代码

2.1.3 安装 Python

建议优先安装 **Python 3.11**。Python 3.12 也可以使用，只要 pandas、scikit-learn、xgboost、openai、pymupdf 等依赖能够正常安装和导入。对于完全新手，Python 3.11 通常更稳妥，因为科研和机器学习库兼容性较成熟。

安装时一定要勾选：

Add Python to PATH

这是最关键的一步。很多新人后面遇到：

python 不是内部或外部命令

就是因为安装时没有勾选这个选项。

安装完成后，打开 VS Code 终端，输入：

```
python --version
```

如果输出类似：

```
Python 3.11.x
```

说明 Python 安装成功。

2.1.4 常见情况一：python 不是内部或外部命令

如果运行：

```
python --version
```

出现：

python 不是内部或外部命令

通常说明 Python 没有加入系统 PATH。

解决方法：

1. 重新运行 Python 安装程序；

2. 勾选 `Add Python to PATH`;
3. 选择 `Modify` 或重新安装;
4. 安装完成后关闭 VS Code;
5. 重新打开 VS Code;
6. 再运行:

```
python --version
```

如果仍然失败, 可以尝试:

```
py --version
```

Windows 有时能识别 `py`, 但识别不了 `python`。

2.1.5 常见情况二: 电脑里有多多个 Python 版本

有时电脑里可能同时存在多个 Python 版本, 导致你以为自己用的是 Python 3.11, 但 VS Code 实际调用的是另一个版本。

解决方法:

在 VS Code 中按:

```
Ctrl + Shift + P
```

搜索:

```
Python: Select Interpreter
```

然后选择你刚安装的 Python 版本。

如果不确定选哪个, 优先选择路径中包含以下内容的解释器:

```
Python311  
Python312
```

2.1.6 安装 VS Code

VS Code 是项目主工作区。它负责:

```
写代码  
打开文件夹  
运行终端命令  
查看 Git 状态  
编辑 Markdown  
运行 Jupyter Notebook
```

建议安装这些 VS Code 插件：

插件	作用
Python	运行 Python 代码
Jupyter	打开 <code>.ipynb</code> 文件
Markdown All in One	写 Markdown 更方便
GitLens	查看 Git 提交记录，可选

2.1.7 正确打开项目文件夹

不要只打开单个 `.py` 文件。正确做法是打开整个项目文件夹。

例如打开：

```
medical-llm-newcomer-challenge/
```

在 VS Code 中选择：

```
File → Open Folder
```

然后选择项目根目录。

这样做的好处是：

```
代码路径更稳定
终端默认位置更清楚
Git 状态能正常显示
文件结构一目了然
```

2.1.8 学会使用 VS Code 终端

在 VS Code 顶部点击：

```
Terminal → New Terminal
```

中文界面一般是：

```
终端 → 新建终端
```

常用命令如下：

命令	作用
<code>pwd</code>	查看当前所在目录

命令	作用
<code>dir</code>	查看当前目录下有哪些文件
<code>cd 文件夹名</code>	进入某个文件夹
<code>cd ..</code>	返回上一级目录
<code>python 文件名.py</code>	运行 Python 脚本
<code>python -m pip install 包名</code>	安装 Python 包

2.1.9 常见情况三： `cd` 报错

如果输入：

```
cd Task1_API
```

出现类似：

```
找不到路径 Task1_API
```

通常有两种原因：

- 1. 当前目录下没有 `Task1_API` 文件夹
- 2. 你已经在 `Task1_API` 文件夹里面了

先输入：

```
pwd
```

查看当前路径。如果当前路径已经是：

```
...\Task1_API
```

就不要再输入：

```
cd Task1_API
```

直接运行：

```
python test_qwen_api.py
```

2.1.10 常见情况四：文件明明存在，但 Python 说找不到

例如出现：

```
FileNotFoundError: data/case_report.pdf
```

常见原因是：

运行脚本时，终端所在目录不对。

解决方法：

1. 输入 `pwd` ；
2. 确认自己是否在正确目录；
3. 如果脚本在 `Task1_API` 里，就进入 `Task1_API` ；
4. 再运行脚本。

更稳的写法是在代码中使用 `pathlib`：

```
from pathlib import Path

base_dir = Path(__file__).resolve().parent
pdf_path = base_dir / "data" / "case_report.pdf"
```

这样可以减少因为终端目录变化导致的路径错误。

2.1.11 安装 Python 第三方库

Python 自带功能有限，科研项目需要安装第三方库。推荐统一使用：

```
python -m pip install 包名
```

例如：

```
python -m pip install pandas
```

不建议只写：

```
pip install pandas
```

因为 `python -m pip` 更能保证安装到当前 Python 环境中。

2.1.12 科研项目推荐一次性安装的依赖

```
python -m pip install pandas numpy matplotlib seaborn scikit-learn xgboost openai python-dotenv pymupdf jupyter
```

各包用途如下：

包名	用途
pandas	读取 CSV、做表格分析
numpy	数值计算
matplotlib	绘图
seaborn	热力图、箱线图
scikit-learn	机器学习建模和评估
xgboost	XGBoost 模型
openai	调用 OpenAI-compatible API
python-dotenv	读取 <code>.env</code> 里的 API Key
pymupdf	读取 PDF 文本
jupyter	运行 Notebook

2.1.13 检查依赖是否安装成功

运行：

```
python -c "import pandas, numpy, sklearn, openai, fitz; print('environment ready')"
```

如果输出：

```
environment ready
```

说明主要依赖安装成功。

2.1.14 常见情况五： `ModuleNotFoundError`

如果出现：

```
ModuleNotFoundError: No module named 'pandas'
```

说明对应的包没有安装。解决：

```
python -m pip install pandas
```

如果出现：

```
No module named 'fitz'
```

注意不是安装 `fitz`，而是安装：

```
python -m pip install pymupdf
```

代码里才写：

```
import fitz
```

2.1.15 常见情况六：安装很慢或失败

如果安装很慢，可能是网络访问国外源较慢。可以换国内镜像，例如清华源：

```
python -m pip install pandas -i https://pypi.tuna.tsinghua.edu.cn/simple
```

一次性安装多个包：

```
python -m pip install pandas numpy matplotlib seaborn scikit-learn -i  
https://pypi.tuna.tsinghua.edu.cn/simple
```

2.1.16 常见情况七：pip 版本过旧

如果看到类似提示：

```
You should consider upgrading via the...
```

可以更新 pip：

```
python -m pip install --upgrade pip
```

如果更新时报权限错误，但项目依赖已经能安装成功，可以暂时不处理。

2.1.17 是否需要虚拟环境？

对完全新手来说，第一天可以不强制使用虚拟环境。但如果要把项目做规范，建议使用。

虚拟环境可以理解为：

```
给每个项目单独建一个 Python 小房间。  
这个项目装的包，不影响其他项目。
```

创建虚拟环境：

```
python -m venv .venv
```

Windows PowerShell 激活：

```
.\.venv\Scripts\Activate
```

如果成功，终端前面会出现：

```
(.venv)
```

然后再安装依赖：

```
python -m pip install pandas numpy matplotlib seaborn scikit-learn xgboost openai python-dotenv pymupdf jupyter
```

如果 PowerShell 不允许激活虚拟环境，出现类似：

```
running scripts is disabled on this system
```

可以运行：

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass
```

然后再次运行：

```
.\.venv\Scripts\Activate
```

2.1.18 Jupyter Notebook 怎么配置？

Jupyter Notebook 是一款交互式编程与文档工具，能把代码、文字、公式、图表和运行结果放在一个文件里。挑战二的数据分析建议使用 Notebook，因为它适合边运行代码边记录结果。

安装：

```
python -m pip install jupyter ipykernel
```

在 VS Code 中新建：

```
heart_failure_analysis.ipynb
```

第一次运行时，VS Code 可能让你选择 Kernel。选择当前 Python 环境即可。如果 Notebook 显示没有 Kernel，重启 VS Code 后重新选择 Kernel。

2.1.19 常见问题速查表

问题	判断方法	解决方法
Python 没装好	<code>python --version</code> 报错	重新安装并勾选 Add Python to PATH
包没装好	<code>ModuleNotFoundError</code>	<code>python -m pip install 包名</code>
PDF 读取失败	<code>No module named fitz</code>	<code>python -m pip install pymupdf</code>
Notebook 没 Kernel	VS Code 提示 Select Kernel	<code>python -m pip install ipykernel</code>
文件找不到	<code>FileNotFoundError</code>	检查当前目录和文件路径
运行错目录	<code>pwd</code> 看路径不对	<code>cd</code> 正确目录
安装太慢	pip 长时间无响应	使用清华源
虚拟环境激活失败	PowerShell 禁止脚本	<code>Set-ExecutionPolicy -Scope Process - ExecutionPolicy Bypass</code>
VS Code 用错 Python	包明明装了还报错	<code>Python: Select Interpreter</code> 重新选择解释器
环境文件误上传	<code>.venv</code> 或 <code>.env</code> 出现在 <code>git status</code>	加入 <code>.gitignore</code>

2.2 如何在国内无痛调用大模型 API

在医疗大模型项目中，我们经常需要让 Python 自动调用大模型完成任务，例如：

从中文病例中抽取医学实体

把病例文本整理成 JSON

辅助生成医学摘要

辅助检查英文论文表达

辅助解释机器学习结果

这里要注意：**网页版 ChatGPT/Claude/Gemini 是手动聊天，API 是让 Python 代码自动和大模型对话。**
例如：题目要求“用 Python 代码调用大模型 API”，就不能只在网页里复制粘贴病例内容，而要真正写脚本调用模型。

2.2.1 API 是什么？

对医学新人来说，可以这样理解：

网页版大模型 = 你手动打开网页和 AI 聊天

API = Python 自动把问题发给 AI，并自动接收 AI 的回答

一个最基本的 API workflow 是：

Python 脚本

→ 读取 API Key

→ 把病例文本和 Prompt 发给大模型

→ 大模型返回结果

→ Python 解析结果

→ 保存为 JSON / TXT / CSV 文件

所以 API 的意义是：**把 AI 能力接入科研代码流程。**

2.2.2 国内常用大模型 API 怎么选？

对于国内医学 AI 新人来说，推荐优先考虑以下 API 平台：

平台 / API	代表模型	国内访问 便利性	中文医学 文本处理	JSON 输出 支持	Python 调用难度	成本/ 免费额度	适合任务	新人推荐 度
阿里云百炼 Qwen	qwen-plus, qwen-turbo, qwen-max	高	高	支持 JSON Mode	低	通常有 新人免 费额 度，价 格较低	中文病例实 体抽取、论 文润色、通 用科研文本 处理	★★★★★
DeepSeek API	deepseek- chat, deepseek- reasoner	较高	高	支持 JSON 输 出	低	价格通 常较低	代码生成、 Debug、推 理分析、文 本结构化	★★★★☆
OpenAI API	GPT 系列	国内使用 不稳定	高	支持结构 化输出	低	成本相 对更高	高质量英文 论文润色、 复杂推理	★★★☆☆
Gemini API	Gemini 系列	国内使用 不稳定	较高	支持结构 化输出	中	有免费 层/低 价层， 但访问 受限	多模态、长 上下文、英 文资料处理	★★★☆☆

平台 / API	代表模型	国内访问 便利性	中文医学 文本处理	JSON 输出 支持	Python 调用难度	成本/ 免费额度	适合任务	新人推荐 度
Claude API	Claude 系列	国内使用 不稳定	高	可通过 Prompt 约束结构	中	成本较高	长文本阅读、论文润色、复杂写作	★★☆☆☆

注：不同平台的模型名称、价格、免费额度和功能会变化。正式使用前，请以平台官网控制台和官方文档为准。本指南重点是帮助新人完成科研训练项目，不做商业采购评估。

2.2.3 选择 API 时应该看哪些指标？

新人选择大模型 API 时，不要只看“哪个模型最强”，而要看它是否适合当前任务。

维度	为什么重要	新人判断方法
国内访问便利性	网络不稳定会导致代码经常超时	国内平台通常更稳
中文医学文本理解	病例、检查、诊断、治疗方案多为中文	优先选中文能力强的平台
JSON 输出能力	挑战一要求保存标准 JSON	优先选择支持 JSON Mode 或结构化输出的平台
Python 调用难度	新人不适合复杂 SDK	优先选择 OpenAI-compatible 接口
成本和免费额度	新人项目调用量不大，但要避免额外费用	优先选有免费额度或低价模型
文档是否清楚	文档越清楚，Debug 越容易	看官方是否提供 Python 示例
是否适合医学任务	医学任务需要稳定、严谨、少幻觉	Prompt 中要强制“不要编造”
是否方便写进项目	代码越通用，展示越清晰	OpenAI-compatible 写法更通用

2.2.4 国内新人优先推荐哪个 API?

对于国内医学 AI 新人项目，推荐优先使用：

```
阿里云百炼 Qwen-Plus
```

原因：

- 1. 国内访问相对稳定；
- 2. 中文病例理解能力较好；
- 3. 题目本身推荐阿里云百炼 Qwen；
- 4. 支持 OpenAI-compatible 接口，代码写法简单；
- 5. 支持 JSON Mode，适合结构化医学实体抽取；
- 6. 新用户通常有免费额度，适合新人挑战。

2.2.5 如何获取 API Key?

基本流程：

```
1. 登录阿里云
2. 进入百炼 Model Studio 控制台
3. 开通服务
4. 进入 API Key / 密钥管理页面
5. 点击创建 API Key
6. 复制并保存 API Key
```

注意：

```
完整 API Key 通常只显示一次。
关闭页面后可能无法再次查看完整内容。
```

所以复制后建议立即保存到本地 `.env` 文件中。

2.2.6 API Key 应该怎么保存?

API Key 类似账号密码，不能公开。

错误写法：

```
api_key = "sk-xxxxxxxxxxxxxxxxxx"
```

这种写法很危险，因为你可能会把代码上传到 GitHub，导致 API Key 泄露。

正确做法是在项目中创建 `.env` 文件：

```
DASHSCOPE_API_KEY=你的真实APIKey
```

然后在 Python 中读取：

```
import os
from dotenv import load_dotenv

load_dotenv()
api_key = os.getenv("DASHSCOPE_API_KEY")
```

2.2.7 常见 API 报错与解决方法

报错	常见原因	解决方法
DASHSCOPE_API_KEY is not set	.env 没被读取	检查 .env 是否和脚本在同一项目目录，变量名是否正确
401 Unauthorized	API Key 错误或无权限	检查 Key 是否复制完整，百炼服务是否开通
model not found	模型名写错	使用 qwen-plus，不要写成 Qwen-Plus 或 qwen_plus
404 Not Found	base_url 写错	检查 base_url 是否正确
JSON 解析失败	模型输出了 Markdown 或解释文字	使用 JSON Mode，并要求 Return ONLY valid JSON
请求太长	输入文本超过上下文限制	分段处理病例文本或精简输入
额度不足	免费额度用完或账号欠费	检查百炼控制台额度和账单
网络错误	本地网络波动	重试，或更换网络环境

2.2.8 不同任务如何选择模型？

任务	推荐模型
API 连通性测试	qwen-plus 或更便宜模型
中文病例实体抽取	qwen-plus
长文本复杂分析	更长上下文模型或分段处理
严格 JSON 输出	支持 JSON Mode 的模型
预算极低的测试	先用低价模型跑通流程，再换正式模型

本项目推荐正式提交用 qwen-plus，因为它在中文医学文本理解、成本和稳定性之间比较平衡。

2.2.9 医疗数据使用 API 时的隐私提醒

即使是新人练习，也要有医疗数据安全意识。

不要上传或发送：

- 真实患者姓名
- 身份证号
- 手机号
- 住院号
- 详细地址
- 可识别影像编号
- 未经脱敏的真实病历

如果项目使用公开病例或公开数据集，也要优先确认：

- 数据来源公开
- 不包含可识别个人身份信息
- 仅用于科研训练和教学

在 Prompt 中也可以加一句：

The provided case text is from a public case report and does not contain identifiable private patient information.

2.2.10 API 调用的最终检查清单

完成 API 配置后，逐项检查：

- ☐ 已开通阿里云百炼
- ☐ 已创建 API Key
- ☐ 已创建 .env 文件
- ☐ .env 中包含 DASHSCOPE_API_KEY
- ☐ 已创建 .env.example
- ☐ .gitignore 已屏蔽 .env
- ☐ 已安装 openai 和 python-dotenv
- ☐ test_qwen_api.py 能正常运行
- ☐ 能看到 Qwen-Plus API test successful
- ☐ 正式脚本没有写死 API Key
- ☐ JSON 输出可以被 Python 正常解析

2.3 如何巧妙利用 ChatGPT / Claude / Gemini 辅助编程

对于医学背景新人来说，使用 AI 辅助编程的关键不是“让 AI 替你完成作业”，而是学会把 AI 当作科研 Copilot：它可以帮你写代码初稿、解释代码逻辑、定位报错、生成图表代码、润色论文和整理提交步骤，但最终结果必须由你自己运行、核查和提交。

核心原则：AI 起草，人类审核。

大模型像“超级聪明但缺乏经验的实习生”。它知识面广、写东西快，但不了解你的项目背景，也可能自信地给出错误答案。因此，AI 适合完成从 0 到 80 分的初稿工作，而你负责从 80 分到 100 分的核查和修正。

2.3.1 为什么医学新人必须学会 AI 辅助编程？

- AI 降低了入门门槛：你可以用自然语言描述任务，让 AI 帮你生成代码、解释报错、拆解流程、生成图表和整理论文。
- 核心技能不是背语法，而是**能清楚描述任务并验证结果**。
- AI 可以帮你完成环境配置、API 调用、JSON 抽取、机器学习分析、论文写作和项目整理，但你必须核查结果。

2.3.2 ChatGPT / Claude / Gemini 辅助编程能力对比

满分 5 星。这里的评分面向“医学科研新人辅助编程”场景，是相对选择参考，不代表所有场景下的绝对强弱。

核心指标	ChatGPT	Claude	Gemini	简要结论
代码质量 & 工程化	★★★★☆	★★★★★	★★★★☆	Claude 最适合从零构建项目、解释代码、组织目录和整理提交流程
上下文长度 & 大项目理解	★★★★★	★★★★★	★★★★☆	ChatGPT、Claude 最适合长文档、长代码和复杂 Notebook 输出
推理、算法和调试	★★★★★	★★★★☆	★★★★☆	ChatGPT 擅长逐步 Debug 和教学式解释，Claude 更适合长日志分析
多模态能力	★★★★☆	★★★★☆	★★★★★	ChatGPT 方便处理截图、PDF、图表和 AI 生图；Gemini 对多模态和 Google 生态辅助较强
性价比	★★★★☆	★★★☆☆	★★★★★	Gemini 性价比通常较高，ChatGPT 中等，Claude 在高频复杂调用下成本较高

2.3.3 使用 AI 辅助编程的正确工作流

不要这样问：

帮我完成挑战二。

这类 Prompt 太大，AI 很容易生成一堆你看不懂、也跑不通的代码。

正确工作流是：

拆任务 → 写最小代码 → 本地运行 → 粘贴报错 → 最小修改 → 再运行 → 保存结果

这套方法的核心是：**每次只验证一个环节**。如果出错，你能知道问题来自哪里。

2.3.4 高质量 Debug Prompt 模板

遇到代码报错时，不要只说“代码错了”。AI 最需要的是上下文。

```
我正在完成医疗大模型新人挑战的第 x 步。

当前目标：
我想实现.....

我已经完成：
1. ....
2. ....

当前目录结构：
粘贴你的目录结构

我运行的命令是：
粘贴命令

完整报错如下：
粘贴完整报错

请你先判断错误类型，再给出最小修改方案。
不要直接重写整个项目。
```

2.3.5 AI 辅助编程的核心经验

经验 1：先让 AI 帮你“理解”，再让它帮你“生成”

新手最容易犯的错误是直接复制 AI 代码。更好的做法是：

- 1. 第一轮：请 AI 解释任务目标和实现思路；
- 2. 第二轮：请 AI 给最小代码；
- 3. 第三轮：自己运行；
- 4. 第四轮：把报错发给 AI；
- 5. 第五轮：让 AI 最小修改；
- 6. 第六轮：让 AI 解释最终代码。

这样你不是在“抄代码”，而是在学会科研编程流程。

经验 2：让 AI 生成“最小可运行版本”

不要一开始就要求功能齐全。例如挑战一不要一次性要求：

```
读取 PDF + 调用 API + 抽取 JSON + 保存结果 + 自动上传 GitHub
```

应该拆成：

- 1. 只测试 API；

2. 只提取 PDF;
3. 只抽取 JSON;
4. 只检查 JSON;
5. 只做提交整理。

最小可运行版本越小，越容易定位问题。

经验 3：不要让 AI 一次性重写整个项目

如果代码只是路径错，AI 却给你重写了 200 行代码，会引入新 bug。你应该明确要求：

请只指出需要修改的位置，不要重写整个脚本。

或者：

请给出最小修改方案，最多修改 3 处。

经验 4：AI 给出的代码必须本地运行

AI 生成的代码看起来正确，不等于真的正确。必须运行：

```
python script_name.py
```

或者在 Notebook 中逐格执行。

判断代码是否可靠，至少看：

1. 是否能运行;
2. 是否生成预期文件;
3. 输出路径是否正确;
4. 结果是否符合题目要求;
5. 是否有敏感信息泄露;
6. 是否和前一步输出一致。

经验 5：模型结果不能让 AI 编

挑战二中，Accuracy、Precision、Recall、F1、AUC 必须来自 Python 实际输出，而不是 AI 猜出来的。

正确做法：

1. 先用 Python 计算模型指标;
2. 再把表格发给 AI;
3. 让 AI 根据真实结果写 Results。

错误做法：

让 AI 自己编一组模型结果。

尤其在医学科研中，任何数据和结论都不能编造。

经验 6：让 AI 帮你发现“隐藏风险”

例如挑战二中，`time` 是随访时间，它与死亡结局强相关。如果直接把它作为输入特征，模型性能可能很好，但可能引入信息泄漏。

可以这样问 AI：

请从医学机器学习角度检查我的特征列表，判断是否存在数据泄漏风险。

这类问题体现你不仅会跑模型，还能理解临床预测任务。

2.3.6 AI 编程的主要风险

风险 1：幻觉

大模型可能生成看起来合理但实际错误的内容。幻觉可能表现为：

1. 编造病例原文没有的既往史；
2. 编造不存在的 Python 参数；
3. 编造不存在的论文引用；
4. 编造模型评价指标；
5. AI 生图中写错 Step 编号或英文单词；
6. 把 GitHub 显示名误认为用户名。

应对方法：

- 所有医学内容回到原文；
- 所有模型结果回到 Python 输出；
- 所有文献引用回到真实 DOI 或 PubMed；
- 所有代码必须本地运行；
- 所有图中文字人工检查。

风险 2：代码能跑，但逻辑错

例如：

- 在划分训练集前对全数据 `fit_transform`，可能造成数据泄漏；
- 把测试集患者当作“新患者”预测，表述上不严谨；
- 把随访时间 `time` 当作纯基线变量，可能夸大模型能力。

所以不能只问：

这段代码能不能跑？

还要问：

这段代码在机器学习流程上是否严谨？
是否存在数据泄漏？
是否符合医学预测任务？

风险 3：AI 生成的代码风格不统一

如果每次让 AI 重写整段代码，项目会变得混乱：

- 变量名不一致；
- 路径不一致；
- 输出文件名不一致；
- 同一功能写多个版本；
- README 和实际代码不一致。

解决方法：

- 固定目录结构；
- 固定文件命名；
- 每次只修改局部；
- 让 AI 遵守已有代码风格。

风险 4：AI Agent 成本和可控性问题

越来越多 coding agent 可以自动编辑文件、运行命令、创建 PR，但它们并不等于“完全可靠的程序员”。对新人来说，不建议一开始就使用完全自动化 agent 修改整个项目。更稳妥的方式是：

1. 先用普通对话式 AI 学会任务；
2. 再手动运行命令；
3. 最后再考虑 agentic coding 工具。

3. 实战篇：遇到跑不通的代码报错，应该用什么样的 Prompt 让 AI 高效 Debug？

对于完全不懂编程的医学背景新人来说，报错并不可怕。真正可怕的是只对 AI 说一句：

代码报错了，怎么办？

这种提问没有上下文，AI 只能猜。高效 Debug 的关键是：

把问题现场完整交给 AI，让它像工程师一样判断错误来源。

3.1 Debug 的核心原则

遇到报错时，不要急着改代码。先做 4 件事：

1. 看清楚当前任务目标；

2. 复制完整报错，不要只截最后一行；
3. 说明当前目录和运行命令；
4. 要求 AI 给“最小修改方案”，不要重写整个项目。

3.2 高效 Debug Prompt 标准模板

我正在完成医疗大模型新人挑战的第 x 步。

当前目标：
我想实现.....

我已经完成：

1.
2.

当前目录：
.....

我运行的命令是：
粘贴你运行的命令

完整报错如下：
粘贴完整报错

- 请你帮我：
1. 判断这是什么类型的错误；
 2. 解释最可能的原因；
 3. 给出最小修改方案；
 4. 告诉我修改后应该运行什么命令验证；
 5. 不要直接重写整个项目。

3.3 实际案例 1: `cd Task1_API` 路径错误

当时目标

我正在挑战一中测试 Qwen-Plus API 是否调用成功，目标是运行：

```
python test_qwen_api.py
```

当时的错误操作

终端当前已经位于：

```
D:\医疗AI+ 科研\内部资料\新手挑战\挑战一\Task1_API
```

但我又输入了：

```
cd Task1_API
python test_qwen_api.py
```

结果出现报错：

```
cd : 找不到路径“D:\医疗AI+ 科研\内部资料\新手挑战\Task1_API\Task1_API”，因为该路径不存在。
```

错误原因

这个错误不是 Python 代码问题，也不是 Qwen API 问题，而是**终端路径问题**。

我当时已经在 Task1_API 文件夹里面，又执行了一次：

```
cd Task1_API
```

但这个子文件夹并不存在，所以报错。

高效 Debug Prompt

我正在完成挑战一：Qwen-Plus API 测试。

当前目标：

运行 test_qwen_api.py，确认 qwen-plus API 是否能正常调用。

当前终端目录：

```
D:\医疗AI+ 科研\内部资料\新手挑战\挑战一\Task1_API
```

我运行的命令是：

```
cd Task1_API
python test_qwen_api.py
```

完整报错如下：

```
cd : 找不到路径“D:\医疗AI+ 科研\内部资料\新手挑战\挑战一\Task1_API\Task1_API”，因为该路径不存在。
Qwen-Plus API test successful.
```

请判断这是 Python 代码问题、API 问题，还是终端路径问题。

请给出最小修改方案，不要重写代码。

AI 的回答

AI 分析后指出：

这是一个终端路径问题，不是 Python 代码问题，也不是 API 问题。

你的终端当前已经在 Task1_API 目录下了（从报错中可以看到完整路径末尾已经是 Task1_API）。你又执行了 cd Task1_API，系统会尝试进入 Task1_API/Task1_API，这个子目录不存在，所以报错。

但好消息是报错下面输出了 "Qwen-Plus API test successful."，说明你的 test_qwen_api.py 其实已经运行成功了。cd 的报错和 Python 脚本的执行是两个独立的操作，cd 失败了但 Python 仍然运行了。

建议：今后运行脚本前先用 pwd 确认当前目录，如果已经在目标目录下，直接 python xxx.py 即可。

最终解决方案

确认当前目录后直接运行脚本，不需要重复 `cd`：

```
# 先用 pwd 确认当前位置
pwd
# 输出末尾已经是 Task1_API → 直接运行
python test_qwen_api.py
```

3.4 实际案例 2：`git add .gitignore` 报 `pathspec` 错误

当时目标

我正在把挑战一的结果整理到本地 Git 项目中，需要添加这些文件：

```
AI_Chat_Records.md
.gitignore
Task1_API/
```

当时的错误操作

我运行了：

```
git add AI_Chat_Records.md .gitignore
```

终端报错：

```
fatal: pathspec '.gitignore' did not match any files
```

错误原因

这个错误的意思是：

```
Git 找不到名为 .gitignore 的文件。
```

原因是我当时还没有真正创建 `.gitignore` 文件，却已经让 Git 去添加它。

这不是 GitHub 问题，也不是网络问题，而是**本地文件不存在**。

高效 Debug Prompt

我正在完成挑战一的项目提交整理。

当前目标：

把 `AI_Chat_Records.md`、`.gitignore` 和 `Task1_API` 目录加入 Git 暂存区。

我运行的命令是：

```
git add AI_Chat_Records.md .gitignore 挑战一
```

完整报错如下：
fatal: pathspec '.gitignore' did not match any files

请判断这个错误是什么意思。
这是 Git 仓库问题、远程仓库问题，还是本地文件不存在？
请给出最小解决步骤。

AI 的回答

AI 分析后指出：

这个错误的意思是 Git 在当前仓库中找不到名为 .gitignore 的文件。

pathspec 是 Git 术语，指文件路径匹配规则。did not match any files 说明你指定的文件在工作区中不存在。这不是 GitHub 问题，也不是网络问题，而是本地文件还没创建。

Git 只能跟踪已经存在的文件。你需要先在项目根目录下手动创建 .gitignore 文件，然后再用 git add 添加到暂存区。

最终解决方案

先创建 .gitignore 文件，再重新执行 git add：

```
# 第一步：在项目根目录创建 .gitignore 文件（可以用 VS Code 直接新建）
# 文件内容至少包含：
# .env
# .venv/

# 第二步：确认文件已存在
ls -la .gitignore

# 第三步：重新添加
git add AI_Chat_Records.md .gitignore Task1_API
```

这个案例的教训是：git add 之前，先用 ls 或 dir 确认文件是否真的存在。

3.5 这两个案例带来的通用 Debug 经验

这两个真实案例都说明：新手遇到的很多“代码问题”，本质上不是算法问题，而是工程基础问题。

问题类型	表现	解决思路
路径问题	找不到路径、FileNotFoundError	用 pwd 和 dir 检查当前位置
文件不存在	pathspec did not match any files	先确认文件是否创建
命令位置错误	在错误文件夹运行脚本	进入正确目录，或用绝对路径
文件名错误	Git 找不到文件	检查大小写、中文、空格、下划线
过度修改	AI 重写整个项目	要求“最小修改方案”

Debug总结

Debug 的关键不是让 AI 猜，而是把“当前目录、运行命令、完整报错、当前目标”一次性给清楚。

4. 进阶篇：如何使用 AI 工具进行高效科研绘图，如何缓解 AI 生图幻觉问题？

AI 绘图工具可以显著提高科研图制作效率，尤其适合生成：

研究流程图
技术路线图
医学 AI 工作流程图
图形摘要
机制概念图
论文插图初稿

但要注意：**AI 生图不是最终成果，而是高质量初稿生成工具。**

正式用于论文、汇报或项目展示前，必须人工检查逻辑、文字、数字和医学准确性。

本项目中，我使用 AI 生成了“心衰患者死亡风险预测模型工作流程图”。这个过程也暴露出 AI 生图的典型问题：图整体很漂亮，但出现了 **Step 编号重复** 和 **英文拼写错误**。因此，AI 科研绘图的关键不是“生成一张好看的图”，而是“生成一张逻辑正确、文字准确、可用于学术表达的图”。

4.1 科研绘图前，先明确图的类型

常见科研图类型如下：

图类型	适合表达什么	本项目是否用到
Workflow flowchart	数据处理、模型训练、预测输出流程	是
Technical roadmap	项目技术路线、模块关系	可选
Graphical abstract	研究主题的视觉摘要	可选
Mechanism diagram	生物机制、病理机制、信号通路	否
Model architecture figure	神经网络结构、算法框架	可选
Statistical plot	ROC、热力图、特征重要性、箱线图	是，但应由 Python 生成
Conceptual medical AI figure	医学 AI 辅助诊疗概念展示	可选

4.2 科研绘图的核心原则：先逻辑，后美观

科研图最重要的是逻辑准确，不是视觉炫酷。

正式生图前，先写清楚 3 件事：

1. 输入是什么？
2. 中间处理步骤是什么？
3. 最终输出是什么？

4.3 推荐的 AI 科研绘图 workflow

建议按下面流程使用 AI 绘图：

- Step 1: 整理研究流程
- Step 2: 压缩成 5-7 个核心模块
- Step 3: 确定图类型和布局
- Step 4: 写详细 Prompt
- Step 5: 生成初稿
- Step 6: 人工检查幻觉
- Step 7: 定点修改
- Step 8: 导入论文或 PPT

不要让 AI 自己决定所有内容。你应该先把核心信息告诉它，尤其是：

研究主题
数据来源
样本量
特征数量
模型名称
关键评价指标
最终输出
图的布局
颜色风格
禁止出现的内容

4.4 如何把复杂研究流程压缩成图中内容

AI 生图最容易出现两个极端：

1. 信息太少，看不出科研流程；
2. 信息太多，图变得拥挤，不可读。

因此要学会“精简但不丢核心”。

4.5 AI 科研绘图 Prompt 的标准结构

一个高质量科研绘图 Prompt 建议包含 7 个部分：

1. **Role:** 让 AI 扮演什么角色
2. **Background:** 研究背景
3. **Core workflow:** 核心流程
4. **Key content:** 必须出现的信息
5. **Layout:** 布局要求
6. **Visual style:** 视觉风格
7. **Avoidance:** 禁止出现的内容

推荐模板：

Role:
You are a senior medical visualization strategist and academic figure-design consultant.

Background:
My research topic is ...

Core workflow:
Input → Processing → Model training → Evaluation → Output

Key content:
Include sample size, clinical features, models, metrics, and final output.

Layout:
Use a clean left-to-right workflow flowchart.
Input on the left, processing in the center, output on the right.

Visual style:
white background, publication-grade, clean sans-serif font, restrained medical color palette.

Avoid:
Do not add unrelated organs, cells, molecules, 3D effects, photos, cartoon style, or invented steps.

4.6 什么是 AI 生图幻觉？

AI 生图幻觉不是只指“画出不存在的东西”，还包括：

文字拼写错误
Step 编号错误
指标数值错误
模型名称错误
箭头逻辑错误
凭空添加不存在的步骤
把流程图画成机制图
添加无关器官、细胞、DNA 或医院场景

对于医学科研图来说，这些错误都可能影响可信度。

4.7 AI 生图幻觉的常见类型

幻觉类型	表现	本项目相关例子
编号幻觉	Step 2 重复、Step 4 跳过	Standardization 被错误标为 Step 2
拼写幻觉	英文单词错误	Applied 被写成 Applierd
数值幻觉	AUC、F1、样本量写错	需要核查 AUC = 0.897、F1 = 0.727
模型幻觉	添加不存在的模型	不能加入 SVM、CNN、Transformer
流程幻觉	增加未做过的步骤	不能加入 external validation
医学幻觉	添加无关器官或机制	不能画成心脏病理机制图
风格幻觉	图像变成海报或卡通	不适合论文
逻辑幻觉	箭头方向错、分支不回流	模型分支应汇入 Evaluation

4.8 如何减少 AI 生图幻觉？

方法 1：Prompt 中明确禁止编造

在 Prompt 中加入：

```
Do not invent missing steps, models, metrics, outcomes, datasets, or clinical claims.
```

中文意思是：

```
不要编造缺失步骤、模型、指标、结局、数据集或临床结论。
```

方法 2：限制图的类型

明确写：

```
This is a workflow flowchart, not a mechanism diagram, not a graphical abstract, and not a disease pathway figure.
```

这样可以减少 AI 添加细胞、分子、器官和通路。

方法 3：使用短标签

AI 对长句文字更容易写错。标签越短，越不容易出错。

不推荐：

```
This step performs comprehensive preprocessing and statistical exploratory analysis of all heart failure clinical records
```

推荐：

Data Preprocessing & EDA

方法 4：数字尽量少而关键

图中只保留必要数字：

299 patients
13 features
AUC values
Best model
F1 score

不要把所有模型指标都塞进图里。

方法 5：生成后让 AI 定点修改

如果图只有一两个小错误，不要说：

重新生成一张。

而要说：

只修改以下两个问题，保持其他布局、颜色、文字和图标不变。

例如：

please only correct:
1. Change “Applierd” to “Applied”.
2. Fix repeated Step 2 numbering.
Keep all other elements unchanged.

这样可以减少“越改越乱”。

4.9 哪些图应该由 AI 生成，哪些图应该由 Python 生成？

不是所有科研图都适合 AI 生图。

图类型	推荐工具	原因
ROC 曲线	Python	必须来自真实模型预测结果
相关性热力图	Python	必须基于真实数据计算
特征重要性图	Python	必须基于真实模型输出
箱线图	Python	必须基于真实数据

图类型	推荐工具	原因
模型性能柱状图	Python	必须准确反映指标
工作流程图	AI 生图 / PowerPoint / BioRender	主要表达流程逻辑
医学 AI 概念图	AI 生图 / BioRender	主要表达研究框架
技术路线图	AI 生图辅助 + 人工修改	结构化逻辑图
机制图	BioRender / Illustrator 优先	医学机制准确性要求高

原则：

凡是依赖真实数据计算的图，用 **Python**。
凡是表达流程、概念、框架的图，可以用 **AI** 生图。

4.10 科研图放进论文前的最终检查清单

- [] 图片文件名规范，例如 `ai_workflow.png`
 - [] 图片保存在 `figures/` 文件夹
 - [] LaTeX 路径正确，例如 `figures/ai_workflow.png`
 - [] 图片分辨率足够清晰
 - [] 图中文字没有拼写错误
 - [] 图中数字与代码结果一致
 - [] 图中模型名称与论文一致
 - [] 图中流程与 **Methods** 一致
 - [] 图注能准确解释图片
 - [] PDF 中图片没有溢出页面